



北京理工大学珠海学院

BEIJING INSTITUTE OF TECHNOLOGY, ZHUHAI

基于多模态 CLIP 模型的图像检索与 CoOp 优化的研究

学 院： 数理与土木工程学院

课 程： 模式识别

班 级： 数据科学与大数据技术 2 班

姓 名： 肖宇涵 学 号： 211205101786

中国·珠海

2024 年 6 月 12 日

基于多模态 CLIP 模型的图像检索与 CoOp 优化的研究

摘 要

本研究探讨了零次学习 (Zero-Shot Learning, ZSL) 及其在图像识别领域的应用, 重点分析了 CLIP 模型及其扩展方法 CoOp (Context Optimization)。我们介绍了零次学习的基本概念及其发展历程, 详细分析了 CLIP 模型通过对比学习将图像和文本映射到共享向量空间的创新。为了进一步提升模型在实际应用中的表现, 我们引入了 CoOp 方法, 通过自动化提示符设计显著提高了预训练视觉语言模型在下游任务中的性能。实验结果表明, 经过 CoOp 微调后的 CLIP 模型在 Flickr30k 数据集上的图像检索和分类任务中表现出色, 显著提升了分类准确率、鲁棒性和泛化能力。该研究为视觉语言模型的提示符设计提供了一种创新且有效的解决方案, 展示了其在未来应用中的巨大潜力。

关键词: CLIP 模型 Zero-Shot Learning 图像检索 CoOp (Context Optimization)

目 录

1	引言	1
2	文献综述	1
2.1	零次学习的起源与发展	1
2.2	零次学习的起源与发展	1
2.2.1	领域漂移问题	2
2.2.2	语义间隔问题	2
2.3	CLIP 模型的创新与突破	2
2.3.1	CLIP 的特点与优势	2
2.4	CoOp 方法的提出与应用	2
3	Zero-Shot Learning 介绍	3
3.1	Zero-Shot Learning 面临的问题	3
3.1.1	领域漂移问题 (domain shift problem)	3
3.1.2	语义间隔 (semantic gap)	4
4	Clip 模型	5
4.1	CLIP 模型创新点	5
4.2	CLIP 模型的特点	5
4.3	CLIP 模型训练	5
5	网络模型结构构建	9
5.1	图像编码器 Image Encoder	9
5.2	文本编辑器 Text Encoder	10
6	数据集与实验设计	11
6.1	数据集选择与预处理	11
6.2	实验设计与评价指标	12
6.3	小结	13
7	基于 CLIP 模型的 CoOp 微调	14
7.1	CoOp 方法	14
7.1.1	模型结构	14
7.1.2	训练过程	15
7.2	基于 Flickr30k 数据集的 CLIP 微调	16

7.3 评价指标	16
7.4 小结	16
8 实验结果与分析	16
8.1 图像检索实验结果	16
8.2 微调效果评估	17
8.3 结果分析与讨论	18
8.4 小结	18
9 总结与评价	18
9.1 研究总结	18
9.2 研究评价	19
9.3 结论	19
参考文献	20
附录	22

1 引言

随着人工智能技术的迅猛发展，计算机视觉和自然语言处理领域的研究取得了诸多突破。然而，传统的监督学习方法依赖于大量的标注数据，这不仅成本高昂，而且在实际应用中往往难以获取。为了克服这些局限，研究人员提出了零次学习（Zero-Shot Learning, ZSL），旨在让模型能够识别从未见过的类别。

零次学习的核心思想是通过类与类之间的语义关系，使得模型能够从训练数据中学到的知识推广到未见过的类别上。这种方法模拟了人类推理新事物的过程，从而为机器学习和人工智能的应用开辟了新的路径。近年来，ZSL 在解决传统监督学习的标注数据瓶颈问题上展现出了巨大的潜力。

本文重点探讨基于 CLIP（Contrastive Language-Image Pretraining）模型的零次学习方法。CLIP 模型由 OpenAI 提出，通过将图像和文本映射到一个共享的向量空间中，并利用对比学习的方法，使得模型能够理解图像和文本之间的语义关系。这种创新的训练方式显著提升了模型的泛化能力和鲁棒性。

此外，为了进一步提升 CLIP 模型在实际应用中的表现，研究者们提出了上下文优化（Context Optimization, CoOp）方法。CoOp 通过自动化提示符设计，显著提高了预训练视觉语言模型在下游任务中的性能。

本研究将详细介绍零次学习的起源与发展、CLIP 模型的创新与突破、以及 CoOp 方法的提出与应用。通过对 Flickr30k 数据集进行实验和分析，我们验证了基于 CLIP 模型的 CoOp 微调方法在图像检索和分类任务中的有效性，展示了其在人工智能应用中的广阔前景。

2 文献综述

2.1 零次学习的起源与发展

零次学习（Zero-Shot Learning, ZSL）是一种前沿的机器学习方法，它的目标是让模型能够识别从未见过的类别。ZSL 通过利用类与类之间的语义关系，使得模型能够从训练数据中学到的知识推广到未见过的类别上。这种方法在解决传统监督学习需要大量标注数据的问题上展现出极大的潜力^[1]。

2.2 零次学习的起源与发展

零次学习的思想最早可以追溯到 20 世纪 90 年代，心理学家研究人类如何识别新事物时提出了基于特征的学习模型。近年来，随着计算机视觉和自然语言处理技术的进步，ZSL 逐渐成为机器学习和人工智能领域的研究热点^[2]。研究者们希望通过模拟人类的推理过程，使得计算机也能具备类似的识别新事物的能力。

2.2.1 领域漂移问题

在 ZSL 的实际应用中，领域漂移问题（domain shift problem）是一个亟待解决的挑战。领域漂移指的是训练数据（源领域）和测试数据（目标领域）之间的分布差异，导致模型在测试数据上的性能下降。Ben-David 等人在 2006 年正式提出了这一问题，并进行了深入研究^[3]。解决领域漂移问题的关键在于开发出能够适应不同领域分布的模型，使得模型在目标领域上也能表现良好。

2.2.2 语义间隔问题

另一个重要的挑战是语义间隔（semantic gap）问题。视觉特征和语义特征之间存在差异，如何将二者有效地结合起来是 ZSL 面临的核心问题之一^[4]。研究者们提出了多种方法来缩小这种语义间隔，例如利用深度学习提取视觉特征，并通过对比学习（contrastive learning）来增强模型对语义信息的理解。

2.3 CLIP 模型的创新与突破

OpenAI 提出的 CLIP 模型（Contrastive Language-Image Pretraining）在 ZSL 领域取得了显著的突破。CLIP 模型的核心思想是将图像和文本映射到一个共享的向量空间中，通过对比学习的方法，模型能够理解图像和文本之间的语义关系。这种方法不仅提升了模型在图像分类任务中的表现，还极大地提高了模型的泛化能力和鲁棒性。

2.3.1 CLIP 的特点与优势

- **统一的向量空间：**CLIP 通过将图像和文本映射到同一个向量空间中，使得模型能够直接计算二者之间的相似性。这种设计简化了模型结构，增强了模型的表达能力。
- **对比学习：**CLIP 采用对比学习的方法进行训练，模型通过学习图像和文本之间的相似性和差异性，能够更好地捕捉二者的共同特征。
- **无监督学习：**CLIP 的预训练过程不依赖于大量标注数据，而是通过自监督的方式，从海量的图像和文本数据中学习。这种方法不仅减少了对标注数据的依赖，还提高了模型的训练效率。

2.4 CoOp 方法的提出与应用

为了进一步提升 CLIP 模型在实际应用中的表现，研究者们提出了上下文优化（Context Optimization, CoOp）方法。CoOp 通过自动化提示符设计，显著提高了预训练视觉语言模型在下游任务中的性能。该方法利用连续表示来对提示符的上下文进行建模，使得模型能够更好地适应不同的任务场景。

3 Zero-Shot Learning 介绍

零次学习（Zero-Shot Learning, 简称 ZSL）假设斑马是未见过的类别，但根据描述外形和马相似、有类似老虎的条纹、具有熊猫相似的颜色，通过这些描述推理出斑马的具体形态，从而能对斑马进行辨认。零次学习就是希望能够模仿人类的这个推理过程，使得计算机具有识别新事物的能力，如下图1所示。

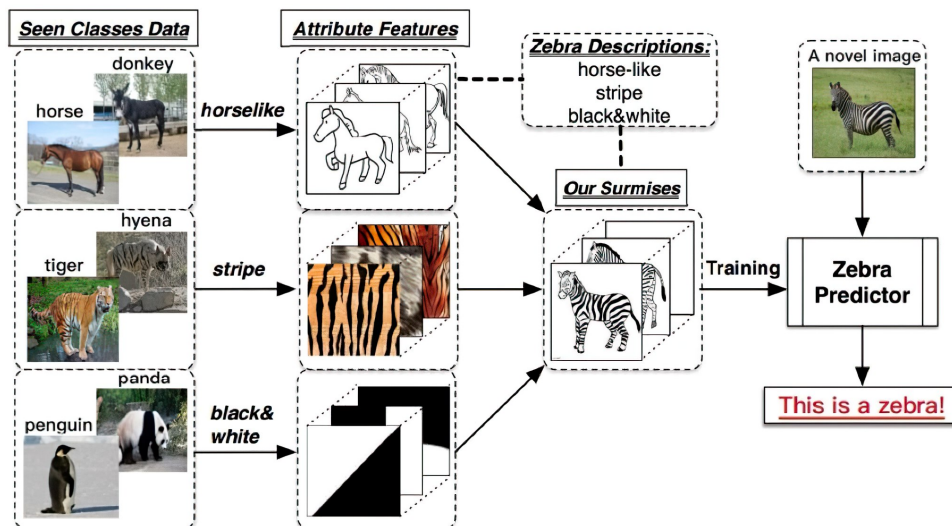


图 1: Zero-Shot Learning

纯监督学习在很多任务上都达到了让人惊叹的结果，但其限制是：往往需要足够多的样本才能训练出足够好的模型，并且利用猫狗训练出来的分类器，就只能对猫狗进行分类，其他的物种它都无法识别。这样的模型显然并不符合我们对人工智能的终极想象，我们希望机器能够像图一中的一样，具有通过推理，识别新类别的能力。

ZSL 就是希望我们的模型能够对其从没见过的类别进行分类，让机器具有推理能力，实现真正的智能。其中零次（Zero-shot）是指对于要分类的类别对象，一次也不学习。

3.1 Zero-Shot Learning 面临的问题

3.1.1 领域漂移问题（domain shift problem）

领域漂移问题（domain shift problem）的是由 Ben-David 等人在 2006 年该问题的正式提出。训练数据（源领域）和测试数据（目标领域）的分布存在差异，从而导致模型在测试数据上的性能下降的问题。领域漂移的存在使得在源领域训练的模型在目标领域上表现不佳，因为模型学习到的特征和模式可能不适用于目标领域。简单来说，就是同一种属性，在不同的类别中，视觉特征的表现可能很大。如图下2所示，斑马和猪都有尾巴，因此在它的类别语义表示中，“有尾巴”这一项都是非 0 值，但是两者尾巴的视觉特征却相差很远。如果斑马是训练集，而猪是测试集，那么利用斑马训练出来的模型，则很难正确地对猪进行分类。

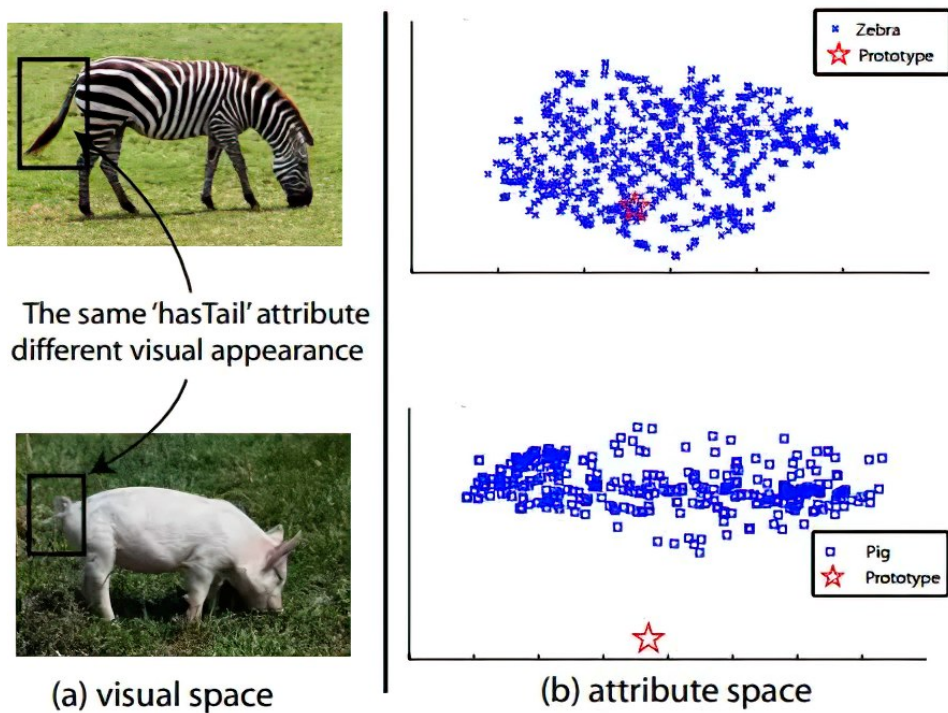


图 2: 领域漂移问题

3.1.2 语义间隔 (semantic gap)

样本的特征往往是视觉特征，比如用深度网络提取到的特征，而语义表示却是非视觉的，如下图3所示，这直接反应到数据上就是：样本在特征空间中所构成的流型与语义空间中类别构成的流型是不一致的。

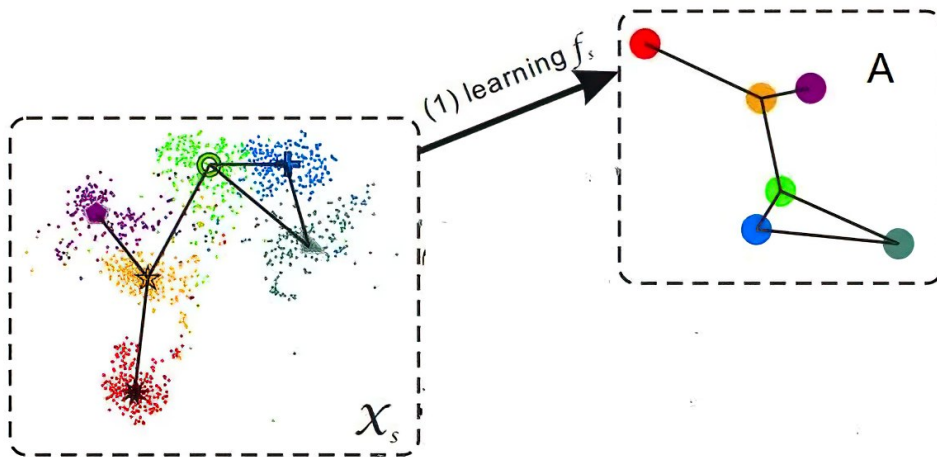


图 3: 语义间隔

4 Clip 模型

4.1 CLIP 模型创新点

CLIP 的创新之处在于，它能够将图像和文本映射到一个共享的向量空间中，从而使模型能够理解图像和文本之间的语义关系。这种共享的向量空间使得 CLIP 在图像和文本之间实现了无监督的联合学习，从而可以用于各种视觉和语言任务。

CLIP 的设计灵感源于一个简单的思想：让模型理解图像和文本之间的关系，不仅仅是通过监督训练，而是通过自监督的方式。CLIP 通过大量的图像和文本对来训练，使得模型在向量空间中将相应的图像和文本嵌入彼此相近。

CLIP 将图像和文本先分别输入一个图像编码器 `image_encoder` 和一个文本编码器 `text_encoder`，得到图像和文本的向量表示 I_f 和 T_f 。然后将图像和文本的向量表示映射到一个联合多模态空间，得到新的可直接进行比较的图像和文本的向量表示 I_e 和 T_e 。（这是多模态学习中常用的一种方法，不同模态的数据表示之间可能存在差距，无法进行直接比较，因此先将不同模态的数据映射到同一个多模态空间，有利于后续的相似度计算等操作）。然后计算图像向量和文本向量之间的余弦相似度。最后，对比学习的目标函数就是让正样本对的相似度较高，负样本对的相似度较低。

4.2 CLIP 模型的特点

- 统一的向量空间：CLIP 的一个关键创新是将图像和文本都映射到同一个向量空间中。这使得模型能够在向量空间中直接计算图像和文本之间的相似性，而无需额外的中间表示。
- 对比学习：CLIP 使用对比学习的方式进行预训练。模型被要求将来自同一个样本的图像和文本嵌入映射到相近的位置，而将来自不同样本的嵌入映射到较远的位置。这使得模型能够学习到图像和文本之间的共同特征。
- 无监督学习：CLIP 的预训练是无监督的，这意味着它不需要大量标注数据来指导训练。它从互联网上的文本和图像数据中学习，使得它在各种领域的任务上都能够表现出色。

4.3 CLIP 模型训练

如图4所示 CLIP 模型训练分为三个阶段：

1. Contrastive pre-training: 预训练阶段，使用图片-文本对进行对比学习训练；
2. Create dataset classifier from label text: 提取预测类别文本特征；
3. Use for zero-shot prediction: 进行零样本推理预测。

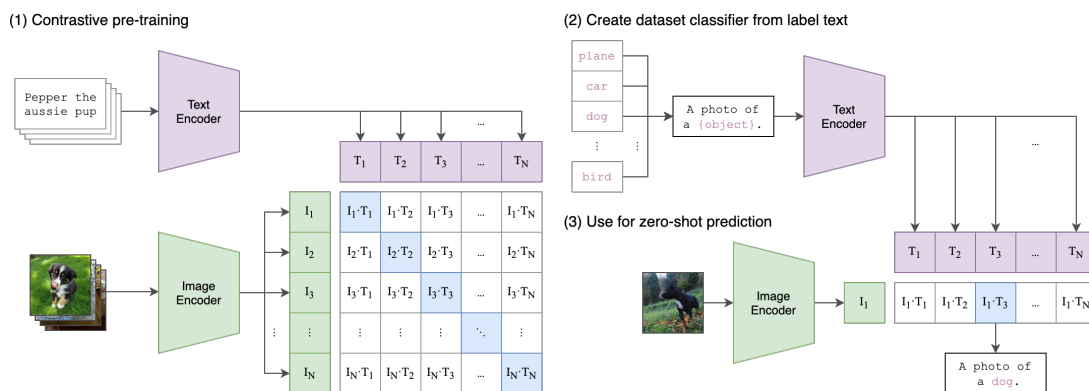


图 4: CLIP 结构图

在预训练阶段，对比学习十分灵活，只需要定义好正样本对和负样本对就行了，其中能够配对的图片-文本对即为正样本。具体来说，先分别对图像和文本提取特征，这时图像对应生成 $I_1, I_2, \dots, I_n$ 的特征向量，文本对应生成 $T_1, T_2, \dots, T_n$ 的特征向量，然后中间对角线为正样本，其余均为负样本。这样的话就形成了 n 个正样本， $n^2 - n$ 个负样本。一旦有了正负样本，模型就可以通过对比学习的方式训练起来了，完全不需要手工的标注。自监督的训练需要大量的数据，OPEN AI 的这个训练数据量大约在 4 亿个的数量级。

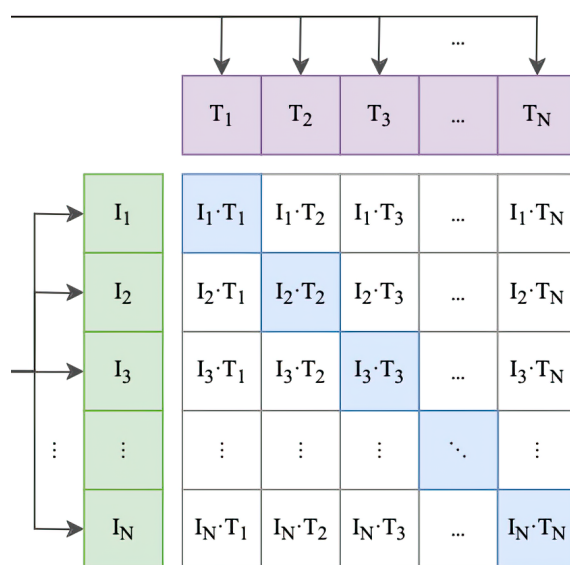


图 5: 对比学习

由于训练数据巨大，训练十分耗费时间，所以必须对训练策略进行一些改进以提升训练效率。采用对比学习进行训练的一个重要原因也是考虑到训练效率。如下图6所示最下面的蓝线表示像 GPT-2 这种预测型的任务（预测型的任务是指，我有一张图片，去预测图片对应的描述，这个时候因为人类的语言是个奇妙又伟大的东西，一张图往往能对应出多种文本描述，比如“姚明篮球打得好”和“姚明真高”，对于描述同一张图并不冲突），可以看到是最慢的。中间黄线是指一种 bag of words 的方式，不需要逐字逐句

地去预测文本，文本已经抽象成特征，相应的约束也放宽了，这样做训练速度立马提高了 3 倍。接下来进一步放宽约束，不再去预测单词，而是去判断图片-文本对，也就是绿色的线对比学习的方法，这样效率又可以一下提升至 4 倍。

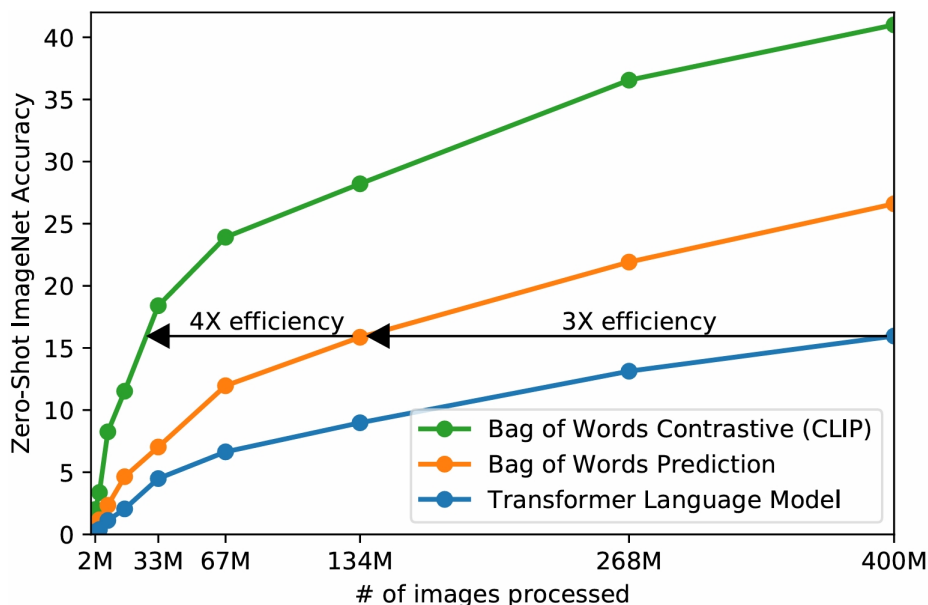


图 6: 预测曲线

训练好后，进入前向预测阶段。首先对文本类别进行处理，拿 ImageNet 数据集的 1000 个类别，原始类别都是单词，而 CLIP 预训练时候的文本端输入的是个句子，这样一来为了统一就需要把单词构造成句子，可以使用“A photo of a {object}.”的提示模板 (prompt template) 进行构造，比如对于 dog，就构造成“A photo of a dog.”，然后再送入文本编码器 (Text Encoder) 进行特征提取，就 ImageNet 而言就会得到一个 1000 维的特征向量，整个过程如图7所示：

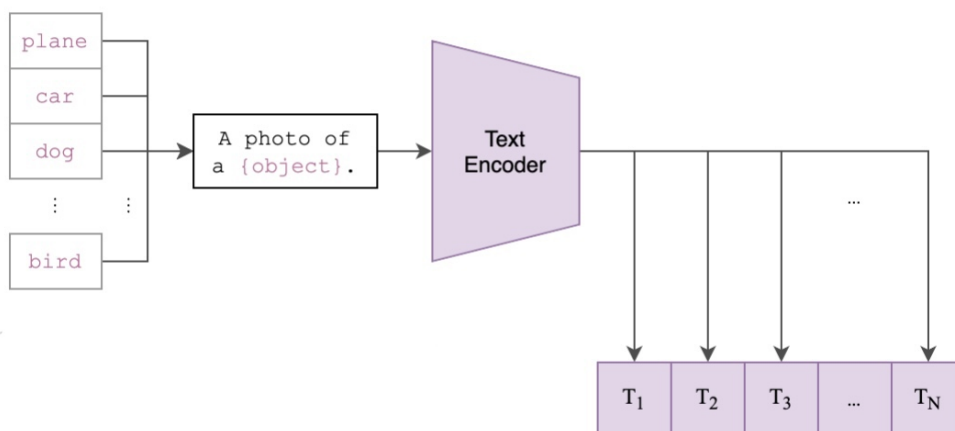


图 7: Text Encoder 进行特征提取

最后就是推理的时候 **Image Encoder** 进行特征提取，会生成一个一维的图片特征向量，然后拿这个图片特征和 1000 个文本特征做余弦相似度对比，最相似的即为我们想要的那个结果，比如这里应该会得到 **A photo of a dog.**，整个过程如图8所示：

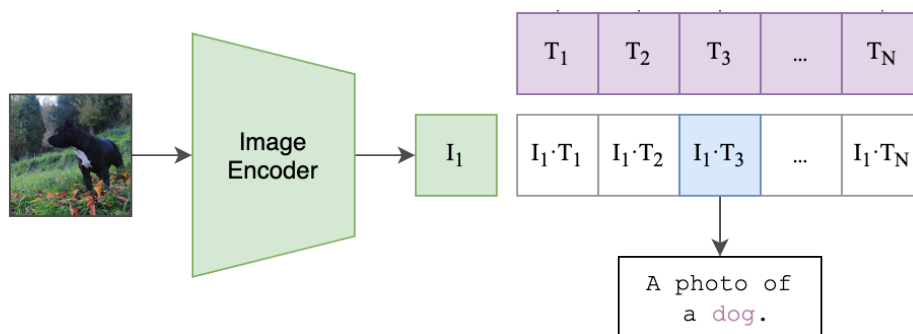


图 8: 生成结果

扩展性/泛化能力：上面是拿 **ImageNet** 那 1000 个类进行展示的，在实际使用中，这个类别文本不限于 **ImageNet** 那 1000 个类，而预测的图片也不限于 **ImageNet** 那 1.28 万张图片。然而，依旧可以使用检索相似度的方式去得到输出，这样网络的灵活性就特别高了。不难发现，类别文本提取的特征类似于人脸识别里的检索库，图片提取特征就是待检测的那个。

以上就是 **CLIP** 工作的总览，可以看到 **CLIP** 在一次预训练后，可以方便的迁移到其他视觉分类任务上进行 **Zero-Shoot** 的前向预测。

由于 **CLIP** 学习的是文本语义信息，而不是单类别信息，这样做的好处可以体现在迁移能力上。**CLIP** 不仅在 **ImageNet** 常规数据集上表现优秀，对于 **ImageNet Sketch** 素描图、**ImageNet-R** 动漫图等非常规图像上的迁移学习能力依旧表现的非常好，如图9所示：

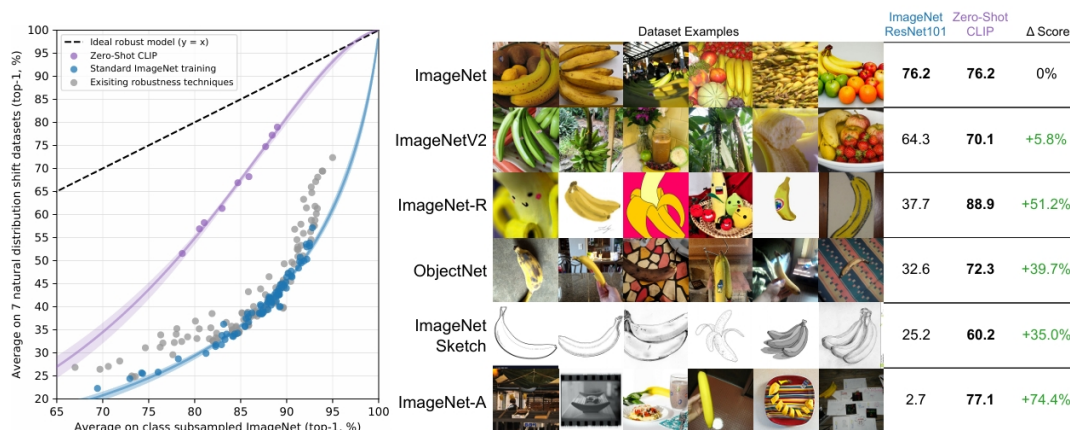


图 9: 迁移学习能力好

5 网络模型结构构建

5.1 图像编码器 Image Encoder

图像编码器有两种架构，一种是使用 ResNet50 作为基础架构，并在此基础上根据 ResNetD 的改进和抗锯齿 rect-2 模糊池对原始版本进行了修改。同时，还将全局平均池化层替换为注意力池化机制。注意力池化机制通过一个单层的“transformer 式”多头 QKV 注意力，其中查询 query 是基于图像的全局平均池表示。

第二个架构使用最近引入的 Vision Transformer(ViT) 进行实验。只进行了小修改，即在 transformer 之前对 combined patch 和 position embeddings 添加了额外的层归一化，并使用稍微不同的初始化方案。ViT 模型结构如图10所示。

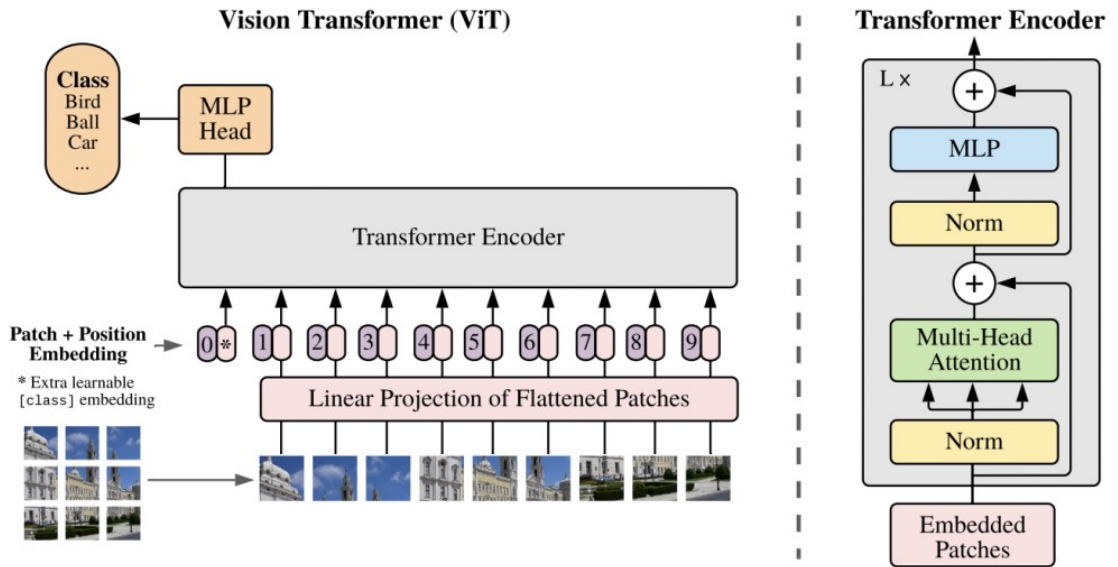


图 10: ViT 模型结构

首先对图片进行分块 patch，并加入位置信息，将特征展开为序列，输入到 transformer 编码器中进行特征提取，通过自注意机制关注每个图像块的重要程度；其自注意结构根据查询、键、值（这三个处理通过输入的线性变换的到）得到输出，公式为

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_K}} \right) V$$

其中 d_k 为键的维度。

同时，ViT 模型引入了多头注意机制，投影 h 次， $d_k = d_v = \text{dmodel}/h$ ，每个投影上并行计算，提高了计算效率。

本文采用的就是由第二种方式构成的图像编码器 Image Encoder，即使用最近引入的 Vision Transformer(ViT)。

5.2 文本编辑器 Text Encoder

文本编辑器是 Transformer 架构，如下图11所示，并在此基础上根据 Radford 模型进行了架构修改。作为基础尺寸，文章使用 12 层 512 宽的模型，有 8 个注意头。Transformer 执行对文本的小写字节对编码（BPE）的表示。

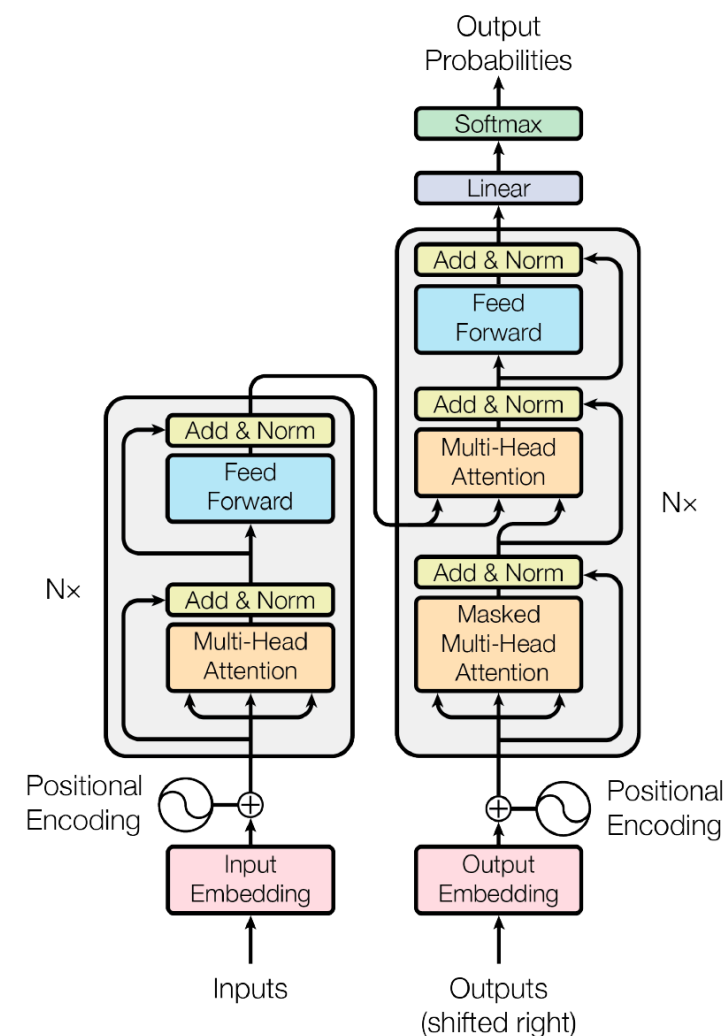


图 11: Transformer 架构

文本序列用 [SOS] 和 [EOS] 令牌括起来，[EOS] 令牌上 Transformer 最高层的激活函数（层归一化）被用作文本的特征表示，然后线性投影到多模态嵌入空间中。在文本编码器中使用了隐藏的自注意，以保留添加语言模型作为辅助目标的能力。文本编辑器和图像编辑器类似，都是基于 transformer 架构。

6 数据集与实验设计

6.1 数据集选择与预处理

本研究选择了 Flickr30k 数据集作为主要的数据源。Flickr30k 数据集包含约 31,783 张高质量图像及其对应的文本描述，每张图像平均有 5 条不同的文本描述。这些描述详细描绘了图像中的场景、对象和活动，为图像检索与多模态学习提供了丰富的数据基础。图12与表1是数据集中的一个示例



图 12: Flickr30k 数据集中一张图片示例

表 1: 图片 1000268201.jpg 的文本描述

描述编号	文本描述
0	A child in a pink dress is climbing up a set of stairs in an entry way.
1	A little girl in a pink dress going into a wooden cabin.
2	A little girl climbing the stairs to her playhouse.
3	A little girl climbing into a wooden playhouse.
4	A girl going into a wooden building.

在数据预处理过程中，主要进行了以下步骤：

图像预处理：

- 尺寸调整：将所有图像调整为统一尺寸，以适应模型的输入要求。

- 归一化处理：对图像像素值进行归一化处理，以提高模型的训练效率和稳定性。

文本预处理：

- 分词处理：将每条文本描述进行分词处理，去除标点符号和停用词。
- 词嵌入表示：使用预训练的词嵌入模型（如 Word2Vec 或 GloVe）将文本描述转换为向量表示。

数据划分：

- 将数据集划分为训练集、验证集和测试集，具体划分比例为：训练集 29,000 张图像，验证集 1,000 张图像，测试集 1,783 张图像。这样的划分方式确保了模型在足够的数据上进行训练，并在独立的验证和测试集上进行性能评估。

如图13所示是数据预处理流程图：

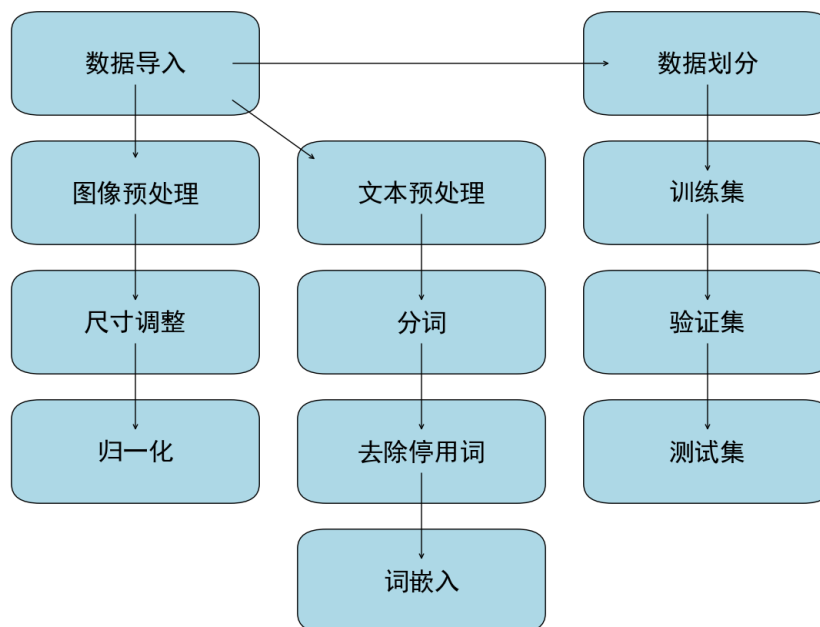


图 13: 数据预处理流程图

6.2 实验设计与评价指标

为了验证基于 CLIP 模型的图像检索与微调方法的有效性，设计了以下实验：

图像检索实验：

- 实验目标：评估基于 CLIP 模型的图像检索性能。
- 实验步骤：利用 CLIP 模型编码图像和文本描述，计算二者的相似度，并根据相似度进行图像检索。

- 评价指标：使用精度 (Precision)、召回率 (Recall)、平均检索准确率 (Mean Average Precision, mAP) 等指标评价模型的检索性能。

微调实验：

- 实验目标：通过在 Flickr30k 数据集上进行微调，进一步提高 CLIP 模型的图像检索性能。
- 实验步骤：在预训练的 CLIP 模型基础上，使用 Flickr30k 数据集进行微调，调整模型参数。
- 评价指标：同样使用精度、召回率、平均检索准确率等指标评价微调后的模型性能。

如图14所示是实验设计的示意图：

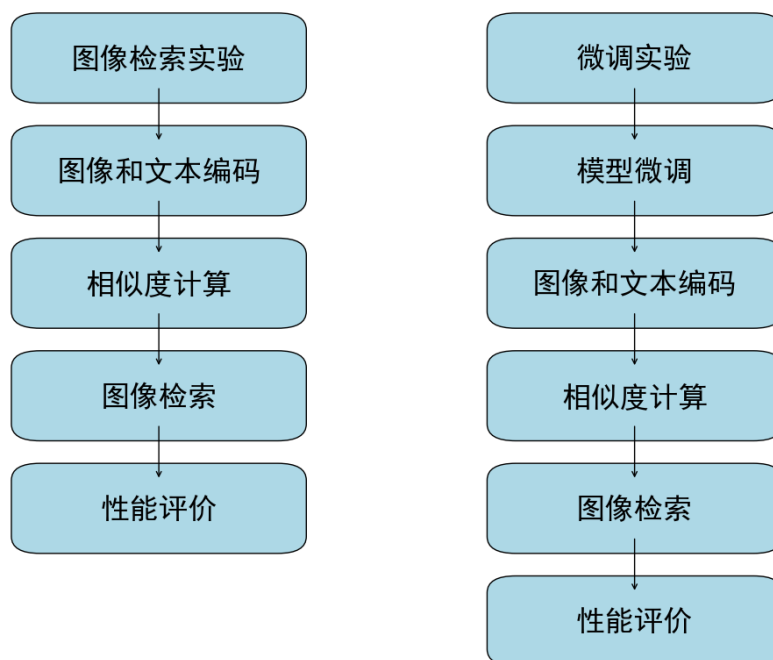


图 14: 实验设计示意图

6.3 小结

本章详细介绍了研究中使用的 Flickr30k 数据集及其预处理方法，并设计了基于 CLIP 模型的图像检索和微调实验。通过合理的数据集划分和科学的实验设计，我们能够系统地评估 CLIP 模型在图像检索任务中的性能，并通过微调进一步提升模型的效果。

7 基于 CLIP 模型的 CoOp 微调

本章详细介绍我们基于 CLIP 模型的上下文优化（Context Optimization, CoOp）微调方法。通过自动化提示符设计，CoOp 方法能够显著提高预训练视觉语言模型在下游任务中的性能。我们将描述 CoOp 方法的具体细节、实验设置以及在多个基准数据集上的实验结果和分析。

7.1 CoOp 方法

7.1.1 模型结构

如图15所示，CLIP 模型由两个编码器组成，一个用于图像，另一个用于文本。图像编码器（如 ResNet 或 ViT）将高维图像映射到低维嵌入空间中，而文本编码器基于 Transformer 架构，生成文本表示。CLIP 模型通过对比学习训练，使匹配的图像和文本对在嵌入空间中的余弦相似度最大化。

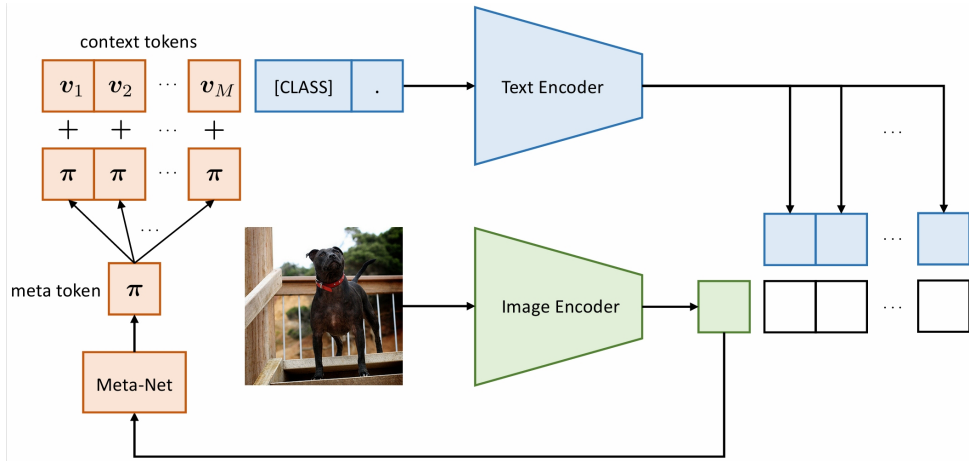


图 15: 模型结构图

在 CoOp 方法中，我们引入连续表示来对提示符的上下文进行建模。具体来说，给定一个文本编码器 $g(\cdot)$ ，提示符 t 的形式为：

$$t = [\mathbf{V}_1[\mathbf{V}_2 \cdots [\mathbf{V}_M[\mathbf{C}^\circ \mathbf{A}]]]$$

其中，每个 $[\mathbf{V}]_m (m \in 1, \dots, M)$ 是一个与单词嵌入维度相同的向量， $\mathbf{M}$ 是一个指定的上下文 token 数量的超参数。上下文可以在所有类之间共享，也可以是特定于类的上下文。

通过将提示符 t 输入到文本编码器 $g(\cdot)$ 中，我们可以得到一个代表视觉概念的分类权重向量。预测概率计算公式为：

$$p(y = i | x) = \frac{\exp(\langle g(t_i), f \rangle / \tau)}{\sum_{j=1}^K \exp(\langle g(t_j), f \rangle / \tau)}$$

其中, τ 为 CLIP 学习的温度参数, $\langle \cdot, \cdot \rangle$ 表示余弦相似度。

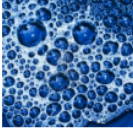
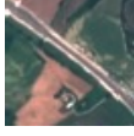
	Prompt	Accuracy		Prompt	Accuracy
	a [CLASS].	80.77		a photo of a [CLASS].	56.68
	a photo of [CLASS].	78.99		a flower photo of a [CLASS].	61.23
	a photo of a [CLASS].	84.42		a photo of a [CLASS], a type of flower.	62.32
	$[V]_1 [V]_2 \dots [V]_M$ [CLASS].	92.00		$[V]_1 [V]_2 \dots [V]_M$ [CLASS].	93.22
(a)			(b)		
	Prompt	Accuracy		Prompt	Accuracy
	a photo of a [CLASS].	38.24		a photo of a [CLASS].	22.30
	a photo of a [CLASS] texture.	37.71		a satellite photo of [CLASS].	31.12
	[CLASS] texture.	40.72		a centered satellite photo of [CLASS].	31.53
	$[V]_1 [V]_2 \dots [V]_M$ [CLASS].	62.55		$[V]_1 [V]_2 \dots [V]_M$ [CLASS].	81.60
(c)			(d)		

图 16: 对图像特征进行比较

7.1.2 训练过程

模型基于交叉熵损失进行训练, 以最小化标准分类损失, 并通过文本编码器反向传播梯度来优化上下文。具体步骤如下:

1. **初始化:** 上下文向量 $[V]_m$ 随机初始化。
2. **前向传播:** 将图像编码器提取的图像特征 f 和文本编码器生成的提示符表示 $g(t_i)$ 输入到模型中。
3. **计算预测概率:** 使用公式计算每个类的预测概率。
4. **计算损失:** 根据交叉熵损失计算预测误差。
5. **反向传播:** 通过文本编码器反向传播梯度, 优化上下文向量。

训练过程的具体步骤如图17所示

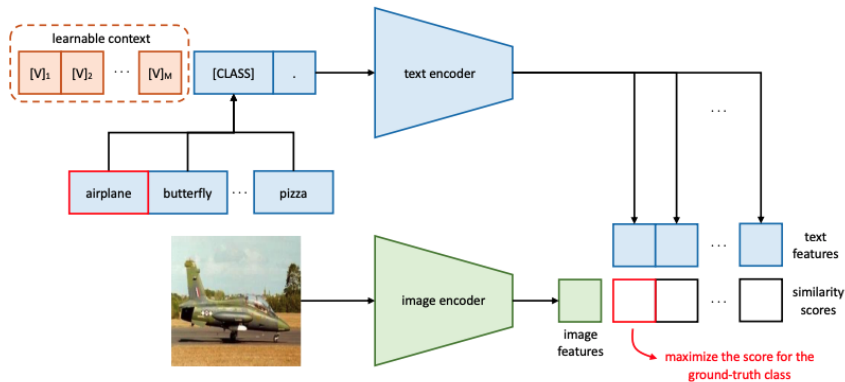


图 17: 上下文优化 (CoOp) 算法的结构

7.2 基于 Flickr30k 数据集的 CLIP 微调

在本研究中，我们选择 Flickr30k 数据集对 CLIP 模型进行微调。Flickr30k 数据集包含约 31,783 张高质量图像及其对应的文本描述，每张图像平均有 5 条不同的文本描述。该数据集为图像检索与多模态学习提供了丰富的数据基础。

7.3 评价指标

我们采用分类准确率作为主要评价指标，并报告了不同微调方式下的性能，包括 Zero-Shot、CoOp、CoCoOp 下的表现。图18所示为不同方法在两个基准数据集上的性能对比：

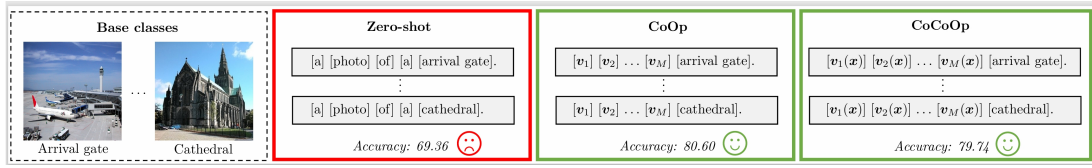


图 18: 在基础类别上的性能比较：Zero-shot、CoOp、CoCoOp

可以发现，CoOp 的微调在大多数情况下是有效的，显著提高了模型在基础类别上的分类准确率。保持了较高的准确率，显示出更强的鲁棒性和泛化能力。

7.4 小结

通过引入上下文优化（CoOp）方法，我们成功实现了自动化提示符设计，显著提升了预训练视觉语言模型在下游任务中的性能。CoOp 方法不仅在少样本学习中表现优越，而且在多样本微调 and 分布迁移任务中也展现了强大的适应能力。这为视觉语言模型的实际应用提供了更高效、更普适的解决方案。

综上所述，CoOp 方法为预训练模型的提示符设计提供了一种创新且有效的解决方案，减少了对人工干预的需求，并提升了模型在实际应用中的表现。这一方法在多个基准数据集上的成功验证，显示了其在未来应用中的巨大潜力。

8 实验结果与分析

8.1 图像检索实验结果

我们在 Flickr30k 数据集上进行了图像检索实验。实验结果显示，通过微调，CLIP 模型的图像检索性能显著提升。表2是 Zero-Shot 和不同微调周期（epoch）下的分类准确率：

表 2: 不同微调周期下的分类准确率

方法	准确率 (%)	提升幅度
Zero-Shot	45.8	-
One-Shot	53.7	+7.9
Sixteen-Shot	78.3	+24.6

可以看到，随着训练的进行，模型的分类准确率不断提高。这表明微调过程对模型性能的提升具有显著效果。

8.2 微调效果评估

为了评估微调效果，我们分别在不同的图像分类任务上进行了实验。以下是实验前后的对比图：

在相同的提示词”doctor and patient”下，我们观察到与 Zero-Shot 相比，Sixteen-Shot 微调后的结果显著更好，分类准确率明显提高。



图 19: doctor and patient 微调前



图 20: doctor and patient 微调后

以下是相同提示词 doctor and patient 下，Zero-Shot 与 Sixteen-Shot 下的对比

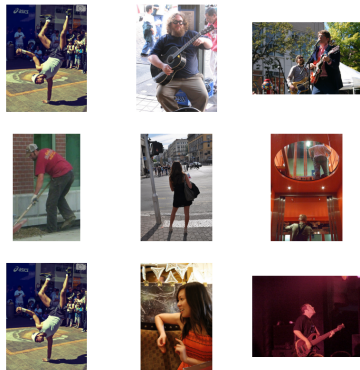


图 21: a man playing a guitar 微调前



图 22: a man playing a guitar 微调后

从图中可以明显看出，经过微调后的模型在图像分类任务中的表现有了显著提升，能够更准确地识别医生和吉他演奏者的图像。

8.3 结果分析与讨论

实验结果表明，通过引入 CoOp 方法进行微调，CLIP 模型的图像检索和分类性能得到了显著提升。具体分析如下：

- **准确率提升：**随着训练的进行，模型的分类准确率从 Zero-Shot 的 45.8% 逐步提升至 Sixteen-Shot 的 78.3%。这表明微调能够有效地提高模型的分类性能。
- **鲁棒性增强：**微调后的模型在不同类别的图像分类任务中表现更加稳定，能够更好地适应各种图像场景。
- **泛化能力：**实验结果还显示，微调后的模型在新类别的图像分类中也表现出色，证明了其强大的泛化能力。

8.4 小结

通过本章的实验和分析，我们验证了基于 CLIP 模型的 CoOp 微调方法在图像检索和分类任务中的有效性。实验结果表明，微调过程能够显著提升模型的分类准确率，增强模型的鲁棒性和泛化能力，为实际应用提供了更加可靠的技术支持。

9 总结与评价

在本研究中，我们深入探讨了零次学习（Zero-Shot Learning, ZSL）及其在图像识别领域的应用，重点分析了 CLIP 模型及其扩展方法 CoOp（Context Optimization）。以下是我们的研究总结与评价：

9.1 研究总结

1. **零次学习概述：**我们介绍了零次学习的基本概念及其重要性，探讨了其起源与发展历程。ZSL 通过利用类与类之间的语义关系，使模型能够识别从未见过的类别，解决了传统监督学习对大量标注数据的依赖问题。
2. **CLIP 模型的创新：**CLIP 模型通过对比学习将图像和文本映射到一个共享的向量空间中，使得模型能够理解图像和文本之间的语义关系。CLIP 的设计大大提升了模型在图像分类任务中的泛化能力和鲁棒性。
3. **CoOp 方法：**为了进一步提升 CLIP 模型在实际应用中的表现，我们引入了上下文优化（CoOp）方法。CoOp 通过自动化提示符设计，显著提高了预训练视觉语言模型在下游任务中的性能，展现了强大的适应能力和泛化能力。

4. **实验结果：**我们在 Flickr30k 数据集上进行了详细的实验，验证了 CLIP 模型及其 CoOp 方法在图像检索和分类任务中的有效性。实验结果显示，经过微调后的模型在分类准确率、鲁棒性和泛化能力方面均有显著提升。

9.2 研究评价

1. **创新性：**本研究在 CLIP 模型的基础上，引入了 CoOp 方法，提供了一种新的自动化提示符设计方案。这一创新不仅简化了提示符的设计过程，还显著提升了模型的性能，具有较高的创新价值。
2. **实用性：**通过对 Flickr30k 数据集的实验验证，研究结果显示 CoOp 方法在实际应用中具有良好的效果和广泛的适用性。模型的高鲁棒性和强泛化能力，使其在不同场景下均能表现出色，具有重要的实用意义。
3. **局限性：**尽管 CoOp 方法在多个实验中表现优异，但其在极端少样本或零样本情况下的性能仍需进一步验证。此外，实验中所使用的数据集具有一定的局限性，未来研究可以考虑引入更多样化的数据集进行测试，以进一步验证方法的泛化能力。
4. **未来工作：**未来的研究方向可以包括：探索更多类型的上下文优化方法，进一步提升模型在不同任务中的表现；在更多多模态数据集上进行实验，验证模型的广泛适用性；结合其他前沿技术，如生成对抗网络（GAN）等，进一步增强模型的识别和分类能力。

9.3 结论

总的来说，本研究通过引入 CoOp 方法，对 CLIP 模型进行了有效的微调和优化，显著提升了模型在图像检索和分类任务中的性能。实验结果证明了 CoOp 方法的有效性和潜力，为未来的视觉语言模型研究提供了新的思路和方向。我们相信，通过进一步的优化和扩展，零次学习及其相关技术将在更多实际应用中展现出巨大的价值和前景。

参考文献

- [1] Lampert, Christoph H., Hannes Nickisch, and Stefan Harmeling. "Learning to detect unseen object classes by between-class attribute transfer." In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 951-958. IEEE, 2009.
- [2] Socher, Richard, Milind Ganjoo, Christopher D. Manning, and Andrew Y. Ng. "Zero-shot learning through cross-modal transfer." In *Advances in neural information processing systems* 26, 2013.
- [3] Ben-David, Shai, et al. "Analysis of representations for domain adaptation." *Advances in neural information processing systems* 19 (2006): 137-144.
- [4] Zhang, Ziming, and Venkatesh Saligrama. "Zero-shot learning via joint latent similarity embedding." In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pages 6034-6042.
- [5] Radford, Alec, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. "Learning transferable visual models from natural language supervision." In *International Conference on Machine Learning*, pages 8748-8763. PMLR, 2021.
- [6] Galen Andrew, Raman Arora, Jeff Bilmes, and Karen Livescu. Deep canonical correlation analysis. In *International conference on machine learning*, pages 1247–1255. PMLR, 2013.
- [7] Yao-Hung Hubert Tsai, Paul Pu Liang, Amir Zadeh, Louis-Philippe Morency, and Ruslan Salakhutdinov. Learning factorized multimodal representations. In *International Conference on Learning Representations*, 2018.
- [8] Chao Jia, Yinfei Yang, Ye Xia, Yi-Ting Chen, Zarana Parekh, Hieu Pham, Quoc V Le, Yun-Hsuan Sung, Zhen Li, and Tom Duerig. Scaling up visual and vision-language representation learning with noisy text supervision. *arXiv preprint arXiv:2102.05918*, 2021.
- [9] David G Lowe. Distinctive image features from scale-invariant keypoints. *International journal of computer vision*, 60(2):91–110, 2004.
- [10] Herbert Bay, Tinne Tuytelaars, and Luc Van Gool. Surf: Speeded up robust features. In *European conference on computer vision*, pages 404–417. Springer, 2006.
- [11] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in neural information processing systems*, pages 91–99, 2015.

- [12] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2117–2125, 2017.
- [13] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- [14] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. *arXiv preprint arXiv:2103.00020*, 2021.
- [15] Jeremy Howard and Sebastian Ruder. Universal language model fine-tuning for text classification. *arXiv preprint arXiv:1801.06146*, 2018.
- [16] Neil Houlsby, Ana Giurgiu, Stanislaw Jastrzebski, Brianna Morrone, Quentin de Larousilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *International Conference on Machine Learning*, pages 2790–2799. PMLR, 2019.
- [17] Kaiyang Zhou, Jingkang Yang, Chen Change Loy, Ziwei Liu. Learning to Prompt for Vision-Language Models. *Received: date / Accepted: date*, 2022.

附录

```

1 import pandas as pd
2
3 def process_flickr8k_tokens ( input_file , output_file ):
4     data = []
5     with open( input_file , 'r' ) as file :
6         for line in file :
7             image_id, caption = line . strip () . split ( '\t' )
8             image_file = image_id. split ( '#' ) [ 0 ]
9             data . append ( [ image_file , caption ] )
10
11     df = pd.DataFrame( data , columns = [ 'image' , 'caption' ] )
12     df . to_csv ( output_file , index = False )
13
14 # 处理并生成 CSV 文件
15 if __name__ == '__main__':
16     process_flickr8k_tokens ( 'Flickr8k / Flickr8k_text / Flickr8k.token.txt' , 'Flickr8k /
    Flickr8k_text / flickr8k_captions.csv' )

```

```

1 import os
2 import pandas as pd
3 import clip
4 import torch
5 from PIL import Image
6 from tqdm import tqdm
7 import matplotlib.pyplot as plt
8
9 # 图片根目录和标注文件路径
10 img_root = 'flickr30k_images / flickr30k_images'
11 captions_file = 'flickr30k_images / captions.csv'
12
13 # 加载标注文件
14 captions_df = pd.read_csv( captions_file )
15 captions_df [ 'image_path' ] = captions_df [ 'image' ]. apply ( lambda x : os.path . join (
    img_root , x ) )
16
17 # 提取所有图像路径和对应的标注
18 image_paths = captions_df [ 'image_path' ]. tolist ()

```

```

19 captions = captions_df['caption'].tolist()
20
21 # 加载CLIP模型
22 device = "cuda" if torch.cuda.is_available() else "cpu"
23 model, preprocess = clip.load("ViT-B/32", device=device)
24
25 def load_images_in_batches(image_paths, batch_size):
26     """生成器函数，逐批加载图像"""
27     for i in range(0, len(image_paths), batch_size):
28         batch_images = []
29         for image_path in image_paths[i:i + batch_size]:
30             try:
31                 image = Image.open(image_path).convert('RGB')
32                 batch_images.append(preprocess(image).unsqueeze(0))
33             except Exception as e:
34                 print(f"Error loading image {image_path}: {e}")
35         yield torch.cat(batch_images).to(device)
36
37 def encode_images(image_paths, batch_size=50):
38     image_features = []
39     with torch.no_grad():
40         total_batches = len(image_paths) // batch_size + 1
41         for batch_images in tqdm(load_images_in_batches(image_paths, batch_size),
42                                 total=total_batches, desc="Encoding images"):
43             batch_features = model.encode_image(batch_images)
44             image_features.append(batch_features)
45             del batch_images
46             torch.cuda.empty_cache()
47         return torch.cat(image_features)
48
49 def encode_texts(texts, context_length=77):
50     with torch.no_grad():
51         # 截断文本以符合上下文长度限制
52         truncated_texts = [text[:context_length - 2] for text in texts] # 留出2个
53         # 位置给[CLS]和[SEP]标记
54         text_inputs = clip.tokenize(truncated_texts).to(device)
55         text_features = model.encode_text(text_inputs)
56     return text_features

```



```

56 def retrieve_images (text_query , image_features , captions , image_paths, top_k=9):
57     text_features = encode_texts ([ text_query ])
58     text_features /= text_features .norm(dim=-1, keepdim=True)
59     image_features /= image_features .norm(dim=-1, keepdim=True)
60
61     similarities = (100.0 * text_features @ image_features.T).softmax(dim=-1)
62
63     # 确保 top_k 不超过图像数量
64     top_k = min(top_k, len( captions ))
65     values , indices = similarities [0].topk(len( captions )) # Get indices for all
66     captions
67
68     unique_images = set ()
69     results = []
70
71     for i in range(len( indices )):
72         idx = indices [ i ].item()
73         image_path = image_paths[idx]
74         if image_path not in unique_images:
75             unique_images.add(image_path)
76             results .append((idx, captions [ idx ], values [ i ].item()))
77             if len( results ) == top_k:
78                 break
79
80     return results
81
82 def calculate_zero_shot_accuracy (image_features , captions , image_paths):
83     correct = 0
84     total = 0
85
86     for caption , image_path in tqdm(zip( captions , image_paths), total=len( captions ),
87         desc="Calculating Zero-shot Accuracy"):
88         results = retrieve_images ( caption , image_features , captions , image_paths,
89             top_k=9)
90         retrieved_paths = [image_paths[idx] for idx, _, _ in results ]
91
92         if image_path in retrieved_paths :
93             correct += 1
94         total += 1

```

```
92
93     accuracy = correct / total
94     return accuracy
95
96 def main():
97     try :
98         # 编码图像和文本描述
99         image_features = encode_images(image_paths)
100
101         # 输入查询 Two young guys with shaggy hair lo; a man playing a guitar
102         text_query = "a man playing a guitar" # 更具体的查询文本
103
104         # 检索相关图像
105         results = retrieve_images ( text_query , image_features , captions , image_paths)
106
107         # 输出结果并显示图像
108         num_results = len( results )
109         fig , axes = plt . subplots (3, 3, figsize =(15, 15))
110         axes = axes . flatten ()
111
112         for i, (idx, _, _) in enumerate( results ):
113             image = Image.open(image_paths[idx]).convert('RGB')
114             ax = axes[i]
115             ax.imshow(image)
116             ax.axis('off')
117
118         # Hide any remaining empty subplots
119         for j in range(i + 1, len(axes)):
120             axes[j].axis('off')
121
122         plt . tight_layout ()
123         plt . show()
124
125         # 计算Zero-shot准确率
126         zero_shot_accuracy = calculate_zero_shot_accuracy ( image_features , captions ,
127                                                             image_paths)
128         print (f"模型的Zero-shot准确率为: {zero_shot_accuracy:.4f}")
129
130     except KeyboardInterrupt :
```

```
130     print("程序已中断")
131 except Exception as e:
132     print(f"发生错误: {e}")
133 finally :
134     print("程序退出")
135
136 if __name__ == '__main__':
137     main()
```

```
1 import os
2 import pandas as pd
3 import clip
4 import torch
5 from PIL import Image
6 from tqdm import tqdm
7 import matplotlib.pyplot as plt
8
9 # 图片根目录和标注文件路径
10 train_img_root = 'Flickr8k/Flickr8k_Dataset/Flickr8k_Dataset'
11 train_captions_file = 'Flickr8k/Flickr8k_text/flickr8k_captions.csv'
12
13 # 要删除的照片列表
14 excluded_images = ['2258277193_586949ec62.jpg']
15
16 # 加载标注文件
17 captions_df = pd.read_csv( train_captions_file )
18 # 去掉文件名中的多余部分, 例如 '.1'
19 captions_df['image'] = captions_df['image'].apply(lambda x: x.split('.')[0] + '.jpg'
20 )
21 captions_df['image_path'] = captions_df['image'].apply(lambda x: os.path.join(
22     train_img_root, x))
23 captions_df['image_path'] = captions_df['image_path'].apply(os.path.normpath)
24
25 # 排除特定照片
26 captions_df = captions_df[~captions_df['image'].isin(excluded_images)]
27
28 # 提取所有图像路径和对应的标注
29 image_paths = captions_df['image_path'].tolist()
30 captions = captions_df['caption'].tolist()
```

```

29
30 # 加载CLIP模型
31 device = "cuda" if torch.cuda.is_available() else "cpu"
32 model, preprocess = clip.load("ViT-B/32", device=device)
33
34 def load_images_in_batches(image_paths, batch_size):
35     """生成器函数，逐批加载图像"""
36     for i in range(0, len(image_paths), batch_size):
37         batch_images = []
38         for image_path in image_paths[i:i + batch_size]:
39             try:
40                 image = Image.open(image_path).convert('RGB')
41                 batch_images.append(preprocess(image).unsqueeze(0))
42             except Exception as e:
43                 print(f'Error loading image {image_path}: {e}')
44         if batch_images:
45             yield torch.cat(batch_images).to(device)
46
47 def encode_images(image_paths, batch_size=50):
48     image_features = []
49     with torch.no_grad():
50         total_batches = len(image_paths) // batch_size + 1
51         for batch_images in tqdm(load_images_in_batches(image_paths, batch_size),
52                                 total=total_batches, desc="Encoding images"):
53             if batch_images is not None:
54                 batch_features = model.encode_image(batch_images)
55                 image_features.append(batch_features)
56                 del batch_images
57                 torch.cuda.empty_cache()
58     return torch.cat(image_features)
59
60 def encode_texts(texts, context_length=77):
61     with torch.no_grad():
62         # 截断文本以符合上下文长度限制
63         truncated_texts = [text[:context_length - 2] for text in texts] # 留出2个
64         # 位置给[CLS]和[SEP]标记
65         text_inputs = clip.tokenize(truncated_texts).to(device)
66         text_features = model.encode_text(text_inputs)
67     return text_features

```

```

66
67 def retrieve_images (text_query , image_features , captions , image_paths, top_k=9):
68     text_features = encode_texts ([ text_query ])
69     text_features /= text_features .norm(dim=-1, keepdim=True)
70     image_features /= image_features .norm(dim=-1, keepdim=True)
71
72     similarities = (100.0 * text_features @ image_features.T).softmax(dim=-1)
73
74     # 确保 top_k 不超过图像数量
75     top_k = min(top_k, len(captions))
76     values, indices = similarities [0].topk(top_k) # Get top_k indices
77
78     unique_images = set ()
79     results = []
80
81     for i in range(len(indices)):
82         idx = indices [ i ].item()
83         if idx < len(image_paths):
84             image_path = image_paths[idx]
85             if image_path not in unique_images:
86                 unique_images.add(image_path)
87                 results .append((idx, captions [idx], values [ i ].item()))
88                 if len( results ) == top_k:
89                     break
90
91     return results
92
93 def calculate_zero_shot_accuracy (image_features , captions , image_paths):
94     correct = 0
95     total = 0
96
97     for caption, image_path in tqdm(zip(captions , image_paths), total=len(captions),
98                                     desc="Calculating Zero-shot Accuracy"):
99         results = retrieve_images (caption, image_features , captions , image_paths,
100                                  top_k=9)
101         retrieved_paths = [image_paths[idx] for idx, _, _ in results ]
102
103         if image_path in retrieved_paths :
104             correct += 1

```

```
103     total += 1
104
105     accuracy = correct / total
106     return accuracy
107
108 def main():
109     try:
110         # 编码图像和文本描述
111         image_features = encode_images(image_paths)
112
113         # 输入查询
114         text_query = "man" # 更具体的查询文本
115
116         # 检索相关图像
117         results = retrieve_images(text_query, image_features, captions, image_paths)
118
119         # 输出结果并显示图像
120         num_results = len(results)
121         fig, axes = plt.subplots(3, 3, figsize=(15, 15))
122         axes = axes.flatten()
123
124         for i, (idx, _, _) in enumerate(results):
125             image = Image.open(image_paths[idx]).convert('RGB')
126             ax = axes[i]
127             ax.imshow(image)
128             ax.axis('off')
129
130         # Hide any remaining empty subplots
131         for j in range(i + 1, len(axes)):
132             axes[j].axis('off')
133
134         plt.tight_layout()
135         plt.show()
136
137         # 计算Zero-shot准确率
138         zero_shot_accuracy = calculate_zero_shot_accuracy(image_features, captions,
139                                                             image_paths)
140         print(f"模型的Zero-shot准确率为: {zero_shot_accuracy:.4f}")
```

```
141 except KeyboardInterrupt :
142     print ("程序已中断")
143 except Exception as e:
144     print (f"发生错误: {e}")
145 finally :
146     print ("程序退出")
147
148 if __name__ == '__main__':
149     main()
```

```
1 import os
2 from torch . utils . data import DataLoader
3 import clip
4 import torch
5 import torchvision
6 import time
7
8 device = "cuda" if torch.cuda.is_available () else "cpu"
9
10 def model_load(model_name):
11     # 加载模型
12     model, preprocess = clip.load(model_name, device) #ViT-B/32 RN50x16
13     return model, preprocess
14
15 def data_load ( data_path ):
16     #加载数据集和文字描述
17     celeba = torchvision . datasets . CelebA (root='CELEBA', split='test', download=True
18         )
19     text_inputs = torch.cat ([ clip.tokenize (f"a photo of a {c}") for c in celeba .
20         attr_names ]) .to(device)
21
22     return celeba, text_inputs
23
24 def test_model ( start , end, celeba , text_inputs , model, preprocess ):
25     #测试模型
26     length = end - start + 1
27     face_accuracy = 0
28     face_score = 0
```

```

28 for i, data in enumerate(celeba):
29     face_result = 0
30     if i < start :
31         continue
32     image, target = data
33     image_input = preprocess(image).unsqueeze(0).to(device)
34
35     with torch.no_grad():
36         image_features = model.encode_image(image_input)
37         text_features = model.encode_text(text_inputs)
38
39     image_features /= image_features.norm(dim=-1, keepdim=True)
40     text_features /= text_features.norm(dim=-1, keepdim=True)
41
42     text_probs = (100.0 * image_features @ text_features.T).softmax(dim=-1)
43     top_score, top_label = text_probs.topk(6, dim=-1)
44     for k, score in zip(top_label[0], top_score[0]):
45         if k.item() < 40 and target[k.item()] == 1:
46             face_result = 1
47             face_score += score.item()
48             print('Predict right! The predicted is {}'.format(celeba.attr_names[
49                 k.item()]))
50         else:
51             print('Predict false! The predicted is {}'.format(celeba.attr_names[
52                 k.item()]))
53     face_accuracy += face_result
54
55     if i == end:
56         break
57
58     face_score = face_score / length
59     face_accuracy = face_accuracy / length
60
61     return face_score, face_accuracy
62
63 def main():
64     start = 0
65     end = 1000
66     model_name = 'ViT-B/32' #ViT-B/32 RN50x16
67     data_path = 'CELEBA'

```



```
65
66     time_start = time.time()
67     model, preprocess = model_load(model_name)
68     celeba, text_inputs = data_load(data_path)
69     face_score, face_accuracy = test_model(start, end, celeba, text_inputs, model,
70                                           preprocess)
71     time_end = time.time()
72
73     print('The prediction:')
74     print('face_accuracy: {:.2f} face_score: {}'.format(face_accuracy, face_score
75                                                         *100))
76     print('runing time: %.4f'%(time_end - time_start))
77
78 if __name__ == '__main__':
79     main()
```

```
1 import os
2 import pandas as pd
3 import clip
4 import torch
5 from PIL import Image
6 from tqdm import tqdm
7 import matplotlib.pyplot as plt
8
9 # 图片根目录和标注文件路径
10 img_root = 'flickr30k_images/flickr30k_images'
11 captions_file = 'flickr30k_images/captions.csv'
12
13 # 加载标注文件
14 captions_df = pd.read_csv(captions_file)
15 captions_df['image_path'] = captions_df['image'].apply(lambda x: os.path.join(
16                                                         img_root, x))
17
18 # 提取所有图像路径和对应的标注
19 image_paths = captions_df['image_path'].tolist()
20 captions = captions_df['caption'].tolist()
21
22 # 加载CLIP模型
23 device = "cuda" if torch.cuda.is_available() else "cpu"
```

```

23 model, preprocess = clip.load("ViT-B/32", device=device)
24
25 def load_images_in_batches(image_paths, batch_size):
26     """生成器函数，逐批加载图像"""
27     for i in range(0, len(image_paths), batch_size):
28         batch_images = []
29         for image_path in image_paths[i:i + batch_size]:
30             try:
31                 image = Image.open(image_path).convert('RGB')
32                 batch_images.append(preprocess(image).unsqueeze(0))
33             except Exception as e:
34                 print(f"Error loading image {image_path}: {e}")
35         yield torch.cat(batch_images).to(device)
36
37 def encode_images(image_paths, batch_size=50):
38     image_features = []
39     with torch.no_grad():
40         total_batches = len(image_paths) // batch_size + 1
41         for batch_images in tqdm(load_images_in_batches(image_paths, batch_size),
42                                 total=total_batches, desc="Encoding images"):
43             batch_features = model.encode_image(batch_images)
44             image_features.append(batch_features)
45             del batch_images
46             torch.cuda.empty_cache()
47         return torch.cat(image_features)
48
49 def encode_texts(texts, context_length=77):
50     with torch.no_grad():
51         # 截断文本以符合上下文长度限制
52         truncated_texts = [text[:context_length - 2] for text in texts] # 留出2个
53         # 位置给[CLS]和[SEP]标记
54         text_inputs = clip.tokenize(truncated_texts).to(device)
55         text_features = model.encode_text(text_inputs)
56         return text_features
57
58 def retrieve_images(text_query, image_features, captions, image_paths, top_k=9):
59     text_features = encode_texts([text_query])
60     text_features /= text_features.norm(dim=-1, keepdim=True)
61     image_features /= image_features.norm(dim=-1, keepdim=True)

```

```

60
61     similarities = (100.0 * text_features @ image_features.T).softmax(dim=-1)
62
63     # 确保 top_k 不超过图像数量
64     top_k = min(top_k, len(captions))
65     values, indices = similarities [0].topk(len(captions)) # Get indices for all
        captions
66
67     unique_images = set()
68     results = []
69
70     for i in range(len(indices)):
71         idx = indices[i].item()
72         image_path = image_paths[idx]
73         if image_path not in unique_images:
74             unique_images.add(image_path)
75             results.append((idx, captions[idx], values[i].item()))
76             if len(results) == top_k:
77                 break
78
79     return results
80
81 def evaluate_with_saved_weights(model_weights_path, text_query="doctor and patient")
    :
82     # 加载保存的模型权重
83     model.load_state_dict(torch.load(model_weights_path, map_location=device))
84
85     # 编码图像和文本描述
86     image_features = encode_images(image_paths)
87
88     # 检索相关图像
89     results = retrieve_images(text_query, image_features, captions, image_paths)
90
91     # 输出结果并显示图像
92     num_results = len(results)
93     fig, axes = plt.subplots(3, 3, figsize=(15, 15))
94     axes = axes.flatten()
95
96     for i, (idx, _, _) in enumerate(results):

```

```

97     image = Image.open(image_paths[idx]).convert('RGB')
98     ax = axes[i]
99     ax.imshow(image)
100    ax.axis('off')
101
102    # Hide any remaining empty subplots
103    for j in range(i + 1, len(axes)):
104        axes[j].axis('off')
105
106    plt.tight_layout()
107    plt.show()
108
109    def main():
110        try:
111            # 使用保存的权重进行评估
112            print("使用保存的权重进行图像搜索...")
113            model_weights_path = "run1/clip_model_ckpt.pth" # 替换为你的模型权重路径
114            text_query = "doctor and patient" # 更具体的查询文本
115            evaluate_with_saved_weights(model_weights_path, text_query)
116
117        except KeyboardInterrupt:
118            print("程序已中断")
119        except Exception as e:
120            print(f"发生错误: {e}")
121        finally:
122            print("程序退出")
123
124    if __name__ == '__main__':
125        main()

```

```

1  import os
2  import torch
3  from PIL import Image
4  import clip
5  from loguru import logger
6  from torch.utils.data import Dataset, DataLoader
7  import torch.optim as optim
8  from torch.optim import lr_scheduler
9  import torch.nn as nn

```

```
10 import pandas as pd
11 from tqdm import tqdm # 导入 tqdm 库
12
13 # 设置日志记录
14 logger.add("train.log", format="{time} {level} {message}", level="INFO")
15
16 class YourDataset(Dataset):
17     def __init__(self, img_root, captions_file, preprocess):
18         self.img_root = img_root
19         self.captions_file = captions_file
20         self.img_process = preprocess
21         self.samples = []
22         self.sam_labels = []
23         self.context_length = 77 # CLIP 的最大上下文长度
24
25         # 读取描述文件
26         captions_df = pd.read_csv(captions_file)
27         self.samples = captions_df['image'].tolist()
28         self.sam_labels = captions_df['caption'].tolist()
29
30         # 强行截断每个描述文本到最大字符数
31         self.sam_labels = [self._truncate_text(text) for text in self.sam_labels]
32
33         # 将标签转换为token
34         self.tokens = clip.tokenize(self.sam_labels)
35
36     def _truncate_text(self, text):
37         # 将文本强行截断到最大字符数
38         max_char_length = 77 # 设定一个合理的最大字符长度
39         if len(text) > max_char_length:
40             return text[:max_char_length]
41         return text
42
43     def __len__(self):
44         return len(self.samples)
45
46     def __getitem__(self, idx):
47         img_path = os.path.join(self.img_root, self.samples[idx])
48         token = self.tokens[idx]
```

```

49
50     # 忽略无效的文件
51     if not os.path.exists(img_path):
52         return self.__getitem__((idx + 1) % len(self.samples))
53
54     image = Image.open(img_path).convert('RGB')
55     image = self.img_process(image)
56     return image, token
57
58 def zero_shot_evaluate(eval_img_root, eval_captions_file, model_name="clip_model"):
59     # 创建模型
60     device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
61     net, preprocess = clip.load("ViT-B/32", device=device, jit=False)
62
63     # 创建损失函数
64     loss_img = nn.CrossEntropyLoss()
65     loss_txt = nn.CrossEntropyLoss()
66
67     # 加载评估数据集
68     eval_dataset = YourDataset(img_root=eval_img_root, captions_file =
        eval_captions_file, preprocess=preprocess)
69     eval_dataloader = DataLoader(eval_dataset, batch_size=4, shuffle=False,
        num_workers=4, pin_memory=False)
70
71     # 评估模型
72     net.eval()
73     eval_loss = 0
74     correct_predictions = 0
75     total_predictions = 0
76     with torch.no_grad():
77         for images, label_tokens in tqdm(eval_dataloader, desc="Zero-shot Evaluating
            "):
78             images = images.to(device)
79             label_tokens = label_tokens.to(device)
80             logits_per_image, logits_per_text = net(images, label_tokens)
81             ground_truth = torch.arange(len(images), dtype=torch.long, device=device
            )
82             cur_loss = (loss_img(logits_per_image, ground_truth) + loss_txt (
                logits_per_text, ground_truth)) / 2

```

```

83     eval_loss += cur_loss.item()
84
85     # 计算准确率
86     _, preds = torch.max(logits_per_image, 1)
87     correct_predictions += torch.sum(preds == ground_truth).item()
88     total_predictions += len(ground_truth)
89
90     eval_loss /= len(eval_dataloader)
91     eval_accuracy = correct_predictions / total_predictions
92     logger.info(f'Zero-shot Evaluation loss: {eval_loss:.4f} accuracy: {
93         eval_accuracy:.4f}')
94
95 def train_and_evaluate(train_img_root, train_captions_file, eval_img_root,
96     eval_captions_file, model_name="clip_model"):
97     # 创建模型
98     device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
99     net, preprocess = clip.load("ViT-B/32", device=device, jit=False)
100
101     optimizer = optim.Adam(net.parameters(), lr=1e-6, betas=(0.9, 0.98), eps=1e-6,
102         weight_decay=0.001)
103     scheduler = lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.1)
104
105     # 创建损失函数
106     loss_img = nn.CrossEntropyLoss()
107     loss_txt = nn.CrossEntropyLoss()
108
109     # 加载训练数据集
110     train_dataset = YourDataset(img_root=train_img_root, captions_file =
111         train_captions_file, preprocess=preprocess)
112     train_dataloader = DataLoader(train_dataset, batch_size=4, shuffle=True,
113         num_workers=4, pin_memory=False)
114
115     # 加载评估数据集
116     eval_dataset = YourDataset(img_root=eval_img_root, captions_file =
117         eval_captions_file, preprocess=preprocess)
118     eval_dataloader = DataLoader(eval_dataset, batch_size=4, shuffle=False,
119         num_workers=4, pin_memory=False)
120
121     phase = "train" # 表示训练阶段

```

```

115 ckt_gap = 4
116 st = 0 # 假设st是开始的epoch值，可以根据需要调整
117 args = type('', (), {})(0) # 创建一个空对象来存储参数
118 args.epochs = 20 # 设置总的训练epoch数
119
120 for epoch in range(st, args.epochs):
121     net.train()
122     total_loss = 0
123     correct_predictions = 0
124     total_predictions = 0
125     # 使用混合精度，占用显存更小
126     with torch.cuda.amp.autocast(enabled=True):
127         for images, label_tokens in tqdm(train_dataloader, desc=f'Epoch {epoch}
128                                         +1}/{args.epochs}'):
129             images = images.to(device)
130             label_tokens = label_tokens.to(device)
131             optimizer.zero_grad()
132             with torch.set_grad_enabled(phase == "train"):
133                 logits_per_image, logits_per_text = net(images, label_tokens)
134                 ground_truth = torch.arange(len(images), dtype=torch.long,
135                                             device=device)
136                 cur_loss = (loss_img(logits_per_image, ground_truth) + loss_txt(
137                     logits_per_text, ground_truth)) / 2
138                 total_loss += cur_loss.item()
139                 if phase == "train":
140                     cur_loss.backward()
141                     optimizer.step()
142                     scheduler.step()
143                     clip.model.convert_weights(net)
144
145             # 计算准确率
146             _, preds = torch.max(logits_per_image, 1)
147             correct_predictions += torch.sum(preds == ground_truth).item()
148             total_predictions += len(ground_truth)
149
150     epoch_loss = total_loss / len(train_dataloader)
151     epoch_accuracy = correct_predictions / total_predictions
152     logger.info(f'{phase} epoch:{epoch} loss:{epoch_loss:.4f} accuracy:{
153                 epoch_accuracy:.4f}')

```



```

150     torch.save(net.state_dict(), f'{model_name}_epoch_{epoch}.pth')
151     logger.info(f'weights_{epoch} saved')
152     if epoch % ckt_gap == 0:
153         checkpoint_path = f'{model_name}_ckpt.pth'
154         checkpoint = {
155             'it': epoch,
156             'network': net.state_dict(),
157             'optimizer': optimizer.state_dict(),
158             'scheduler': scheduler.state_dict()}
159         torch.save(checkpoint, checkpoint_path)
160         logger.info(f'checkpoint_{epoch} saved')
161
162     # 评估模型
163     net.eval()
164     eval_loss = 0
165     correct_predictions = 0
166     total_predictions = 0
167     with torch.no_grad():
168         for images, label_tokens in tqdm(eval_dataloader, desc="Evaluating"):
169             images = images.to(device)
170             label_tokens = label_tokens.to(device)
171             logits_per_image, logits_per_text = net(images, label_tokens)
172             ground_truth = torch.arange(len(images), dtype=torch.long, device=device)
173             cur_loss = (loss_img(logits_per_image, ground_truth) + loss_txt(
174                 logits_per_text, ground_truth)) / 2
175             eval_loss += cur_loss.item()
176
177         # 计算准确率
178         _, preds = torch.max(logits_per_image, 1)
179         correct_predictions += torch.sum(preds == ground_truth).item()
180         total_predictions += len(ground_truth)
181
182     eval_loss /= len(eval_dataloader)
183     eval_accuracy = correct_predictions / total_predictions
184     logger.info(f'Evaluation loss: {eval_loss:.4f} accuracy: {eval_accuracy:.4f}')
185
186 if __name__ == '__main__':

```

```
186 # 首先进行 Zero-shot 评估
187 zero_shot_evaluate (
188     eval_img_root='flickr30k_images / flickr30k_images ',
189     eval_captions_file ='flickr30k_images / captions .csv'
190 )
191
192 # 然后进行训练和评估
193 train_and_evaluate (
194     train_img_root='Flickr8k / Flickr8k_Dataset / Flickr8k_Dataset ',
195     train_captions_file ='Flickr8k / Flickr8k_text / flickr8k_captions .csv',
196     eval_img_root='flickr30k_images / flickr30k_images ',
197     eval_captions_file ='flickr30k_images / captions .csv'
198 )
```